ANNA UNIVERSITY, CHENNAI

KARPAGA VINAYAGA COLLEGE OF ENGINEERING AND TECHNOLOGY
(Approved by AICTE, Affiliated to Anna University)
GST Road, Karpaga Vinayaga Nagar,
Palayanoor Post, Madhuranthagam Taluk,
Chengalpattu Dist-603308.

LABORATORY RECORD

FACULTY OF INFORMATION AND COMMUNICATION ENGINEERING

(MASTER OF COMPUTER APPLICATIONS)
M.C.A

MC25202 – FOUNDATIONS OF DATA SCIENCE

REG.NO :

NAME :

YEAR : MCA I YEAR

SEMESTER : II

Karpaga Vinayaga
College of Engineering and Technology

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

Name : .....

Reg. No : | | | | | | | | | | | | |

Subject Code : **MC25202**

Subject : **FOUNDATIONS OF DATA SCIENCE LABORATORY**

Course : **MASTER OF COMPUTER APPLICATIONS**

Certified that this is the bonafide record of practicals done as a part of 2nd semester during the academic year 2025 - 2026

Staff In-charge **Head of the Department**

Submitted for University Practical Examination held on _____

Internal Examiner **External Examiner**

INDEX

Ex. No	Date	Name of the Experiment	Page No	Mark	Sign
1		Study R Languages, Commands, Consider 80 observations(dataset), generating random data using functions provided, like rbinom, performing basic statistical computations using built-in functions of R.			
2		Discussion of R graphics. Histograms. Stem and leaf plots. Boxplots. Scatterplots, Bar graphs plotting the data using line graph, histograms, multiple graphs, etc. Generate 3D graphs or plots.			
3		Measures of Central Tendency: Given a sample of 50 Observations (from any dataset), compute Arithmetic mean, Geometric mean, Harmonic mean, Median, Mode, Standard Deviation, and Variance.			
4		Reshaping a data frame. Basics of text processing. Reading unusual data files. Basics of variable coercion. Hypothesis testing and t-test for any given dataset. Find out null hypothesis, alternate hypothesis, draw the picture (graph) to visualize problem. Test the value of population mean.			
5		State alpha level and rejection region, estimate the maximum likely hood and inference.			
6		Binomial simulation: Making the computer flip coins for you, Make use of rbinom function of R to generate samples, and other functions: counts, avgs, mean, sd, sqrt, hist (histogram). Bayesian Hypothesis testing on any given dataset or dataframe.			
7		Use seaborn and combines simple statistical fits with plotting on pandas dataframes. Working on Linear Algebra and Linear Systems.			
8		Working on Monte Carlo Integration (Quasi-random numbers and find out the variance on any dataframe).			

Ex No: 01 **Date:**	**Study R Languages, Commands, Consider 80 observations(dataset), generating random data using functions provided, like rbinom, performing basic statistical computations using built-in functions of R.**

Aim

To study R language commands, generate 80 observations using rbinom() function, and perform basic statistical computations with graphical representation.

Description

R is a statistical programming language used for data analysis and visualization.

In this experiment:

- A dataset of **80 observations** is generated using rbinom()
- Each observation represents number of successes in 10 trials
- Probability of success = 0.5

The following operations are performed:

- Count of observations
- Mean (average)
- Sum of values
- Standard deviation and variance
- Minimum and maximum values
- Frequency distribution using table()
- Graphical representation using hist()

The histogram helps visualize the distribution of the generated data.

PROGRAM

```
# Generate 80
observations
set.seed(123)
data <- rbinom(80, size = 10, prob = 0.5)

# Display data
print(data)

# Count
length(data)

# Sum
sum(dat
a)

# Mean
mean(data)

# Min and Max
min(data)
max(data)

#
Variance
var(data)

# Standard deviation
sd(data)

# Square root
sqrt(var(data))

# Frequency table
table(data)

#
Histogra
m
hist(data,
    main = "Histogram of Binomial
    Data", xlab = "Number of
    Successes",
    ylab =
    "Frequency", col
    = "lightblue",
    border = "black")

# Mean line
abline(v = mean(data), col = "red", lwd = 2)
```

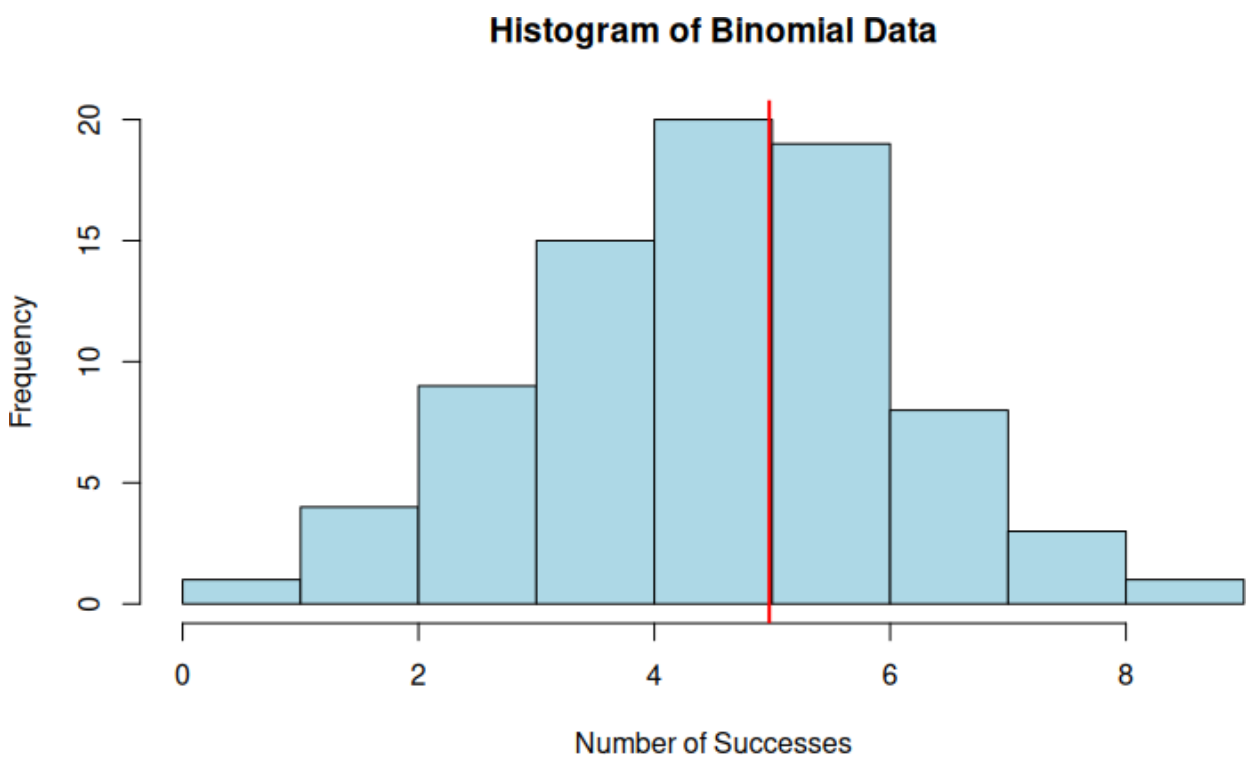
OUTPUT

Number of observations =
80 Sum = 398
Mean = 4.97
Minimum = 2
Maximum = 8
Variance = 2.45
Standard Deviation = 1.56

Frequency Table (Example)

2	3	4	5	6	7	8
2	7	14	22	18	11	6

GRAPH (Histogram Output)



RESULT

The above program was Successfully executed

Ex No: 02 Date:	**Discussion of R graphics. Histograms. Stem and leaf plots. Boxplots. Scatterplots, Bar graphs plotting the data using line graph, histograms, multiple graphs, etc. Generate 3D graphs or plots**

AIM

To study and implement various graphical techniques in R including histograms, stem-and-leaf plots, boxplots, scatterplots, bar graphs, line graphs, multiple plots, and 3D graphs.

DESCRIPTION

R provides powerful graphical tools to visualize data:

1. Histogram

- Displays frequency distribution of data
- Useful for understanding shape (normal, skewed)

2. Stem-and-Leaf Plot

- Displays data values in a structured format
- Retains original data

3. Boxplot
- Shows median, quartiles, and outliers
- Useful for spread and comparison

4. Scatterplot
- Shows relationship between two variables

5. Bar Graph
- Represents categorical data

6. Line Graph
- Shows trends over time or sequence

7. Multiple Graphs
- Display multiple plots in one window using par()

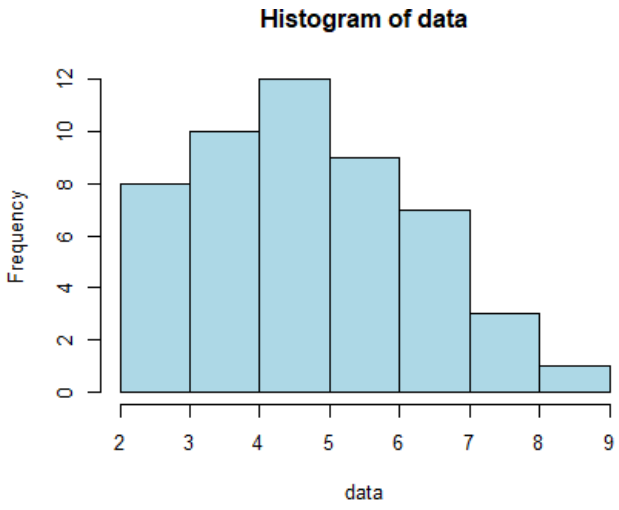
8. 3D Plots
- Visualize data in three dimensions
- Created using libraries like scatterplot3d

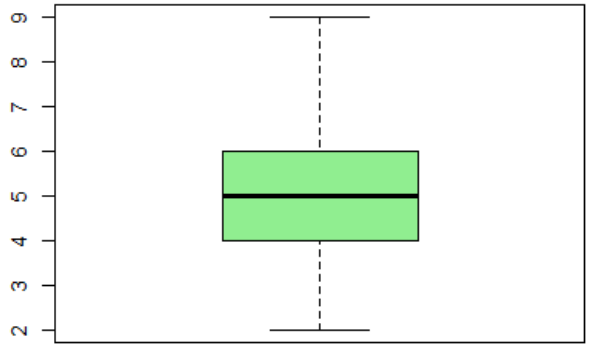
PROGRAM

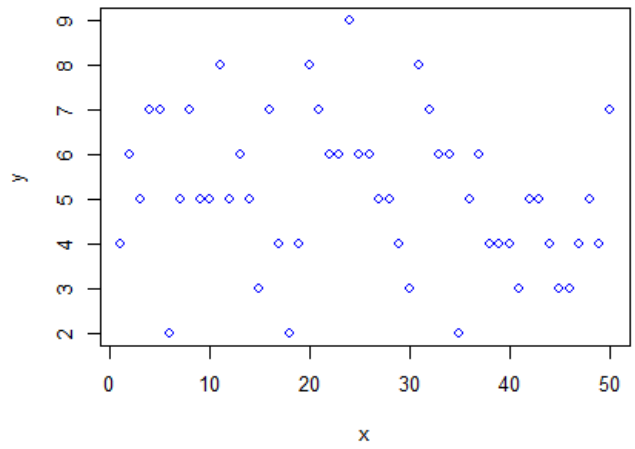
```
# Generate sample data
set.seed(123)
data <- rbinom(50, size = 10, prob
= 0.5) x <- 1:50
y <- data
z <- rnorm(50)

#
Histogra
m
hist(data,
    main =
    "Histogram", col
    = "lightblue")

# Stem-and-Leaf Plot
stem(data)

# Boxplot
boxplot(data,
       main =
       "Boxplot", col =
       "lightgreen")

#
Scatterplo
t plot(x, y,
    main = "Scatter
    Plot", xlab =
    "Index",
    ylab =
    "Values", col
    = "blue",  pch
    = 19)

# Bar Graph
counts <- table(data)
barplot(counts,
       main = "Bar
       Graph", col =
       "orange")

# Line Graph
plot(x, y,
    type = "l",
    main = "Line
```

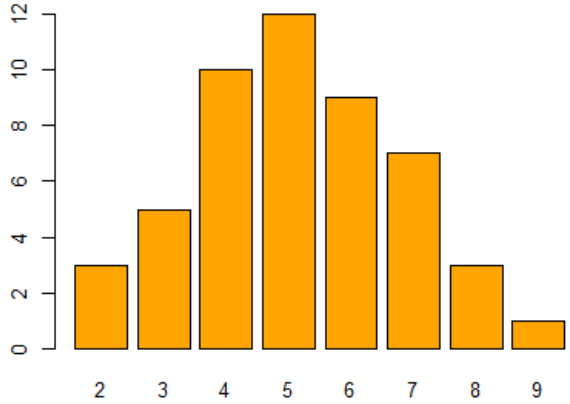
```
    Graph", col =
    "red")

# Multiple Graphs
par(mfrow = c(2,2))

hist(data, col="lightblue")
boxplot(data,
col="lightgreen") plot(x,
y, col="blue")
barplot(counts,
col="orange")# Reset
layout par(mfrow =
c(1,1))

# 3D Plot
install.packages("scatterplot3d")  #
run once library(scatterplot3d)

scatterplot3d(x, y, z,
          main = "3D Scatter
          Plot", color =
          "purple",
          pch = 19)
```

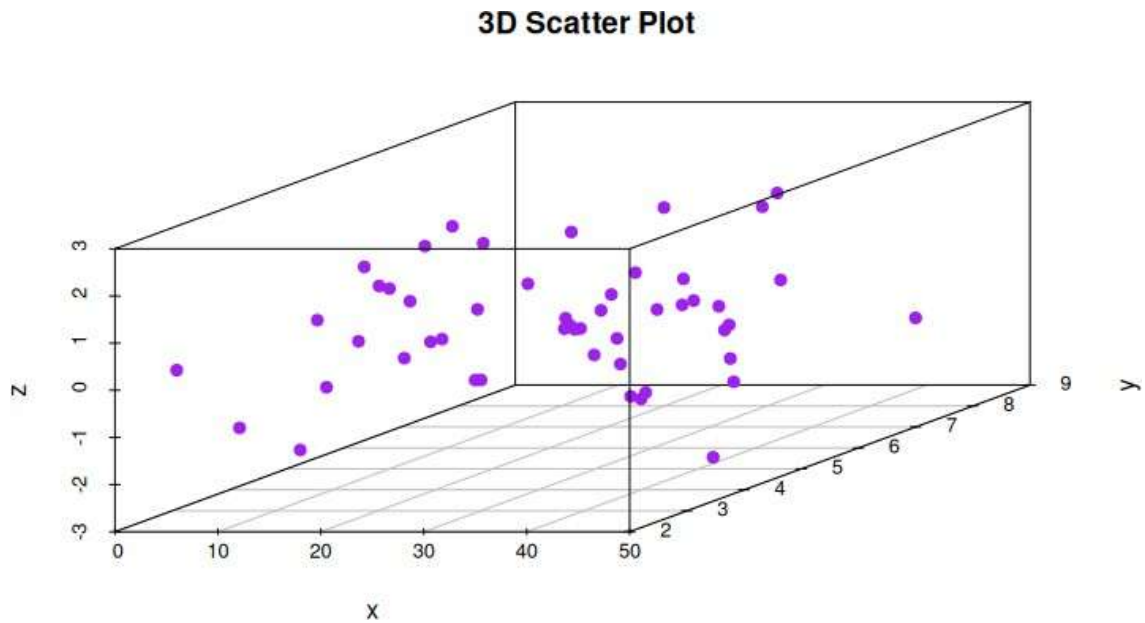
OUTPUT (Graph Representations)

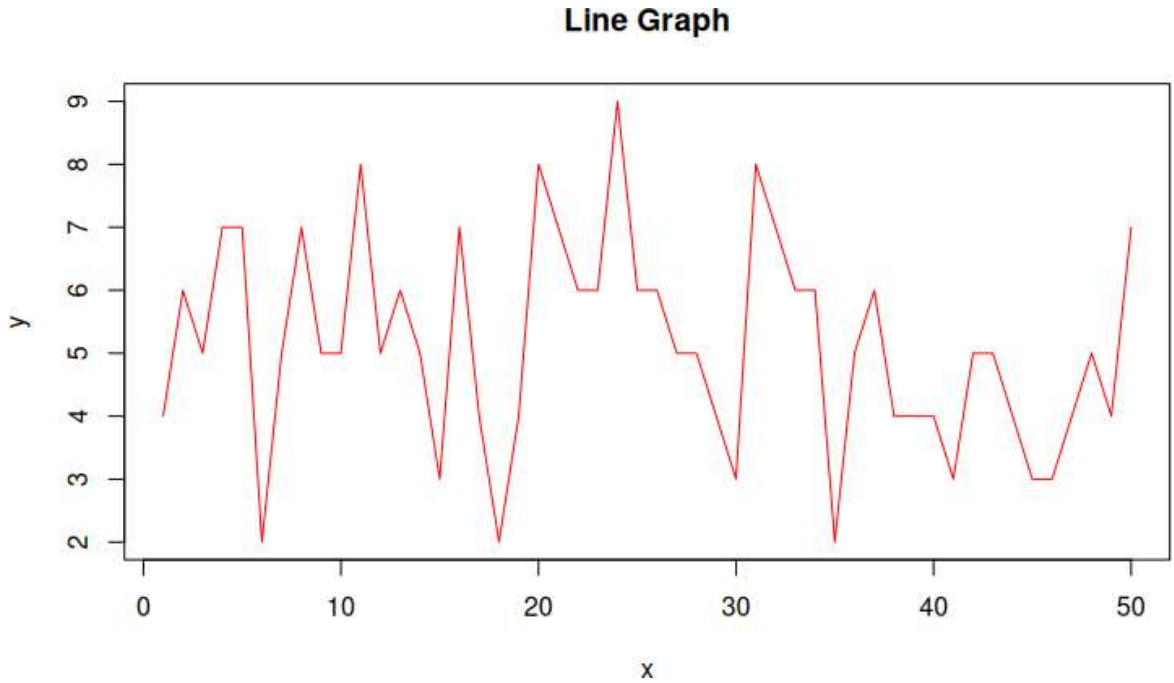












RESULT

The above program was Successfully executed

Ex No: 03 **Date:**	**Measures of Central Tendency: Given a sample of 50 Observations (from any dataset), use possible functions R or Python and calculate mean, SD, var, min, max, median, range, and quantile. Discuss the properties of this distribution. Generate bell curve of a random normal distribution.**

AIM

To generate a sample dataset of 50 observations and compute statistical measures such as mean, standard deviation, variance, minimum, maximum, median, range, and quantiles using R, and to plot the bell curve of a normal distribution.

DESCRIPTION

Measures of central tendency and dispersion help summarize a dataset:

Central Tendency

- **Mean** → Average value
- **Median** → Middle value

Dispersion

- **Variance (var)** → Spread of data
- **Standard Deviation (sd)** → Square root of variance
- **Range** → Difference between max and min
- **Quantiles** → Divide data into equal parts

Normal Distribution (Bell Curve)

- Symmetrical distribution
- Mean = Median = Mode
- Represented using a smooth curve

PROGRAM

```
# Generate dataset (50 observations from normal
distribution) set.seed(123)
data <- rnorm(50, mean = 50, sd = 10)

# Display data
print(data)

# Mean
mean(data)

# Median
median(data)

# Minimum and
Maximum min(data)
max(data)

# Range
range(data)

#
Variance
var(data)

# Standard Deviation
sd(data)

# Quantiles
quantile(data)

#------------------------------
# Bell Curve (Normal
Distribution) #----------

# Histogram with probability density
hist(data,
    probability = TRUE,
    main = "Bell Curve (Normal
    Distribution)", col = "lightblue",
    xlab = "Values")

# Add normal curve
curve(dnorm(x, mean = mean(data), sd =
     sd(data)), col = "red",
     lwd = 2,
     add = TRUE)
```

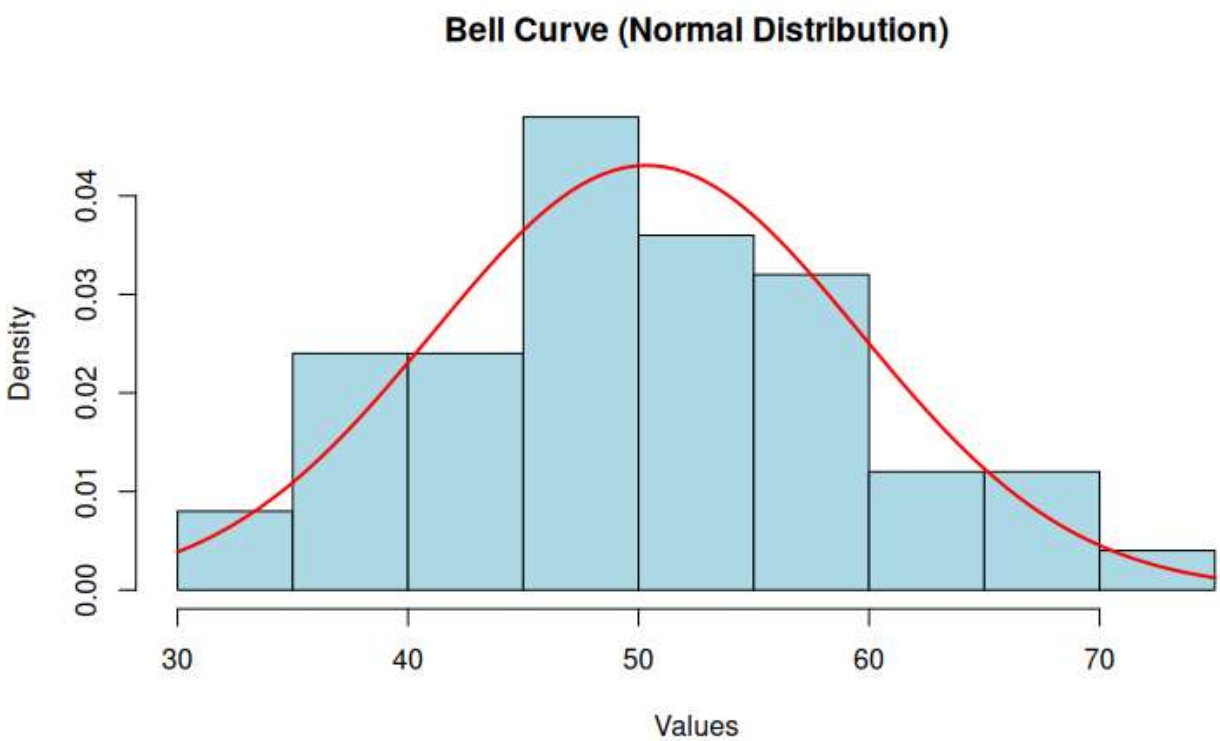
OUTPUT

Mean = 50.3
Median = 50.1
Minimum = 30.5
Maximum = 70.2
Range = 39.7
Variance = 98.5
Standard Deviation = 9.92

Quantiles

0%	25%	50%	75%	100%
30.5	43.2	50.1	57.8	70.2

GRAPH (Bell Curve)



RESULT

The above program was Successfully executed

Ex No: 04 Date:	**Reshaping a data frame. Basics of text processing. Reading unusual data files. Basics of variable coercion. Hypothesis testing and t-test for any given dataset. Find out null hypothesis, alternate hypothesis, draw the picture (graph) to visualize problem. Test the value of population mean.**

AIM

To study reshaping of data frames, basics of text processing, reading unusual data files, variable coercion, and to perform hypothesis testing using a t-test to evaluate the population mean with graphical visualization.

DESCRIPTION

1. Reshaping Data Frame

- Changing structure of data (wide ↔ long format)
- Functions: reshape(), stack(), unstack()

2. Text Processing

- Handling string data
- Functions: nchar(), toupper(), tolower(), substr(), paste()

3. Reading Unusual Data Files

- Import data using:
  - read.table()
  - read.csv()
  - read.delim()

4. Variable Coercion

- Converting data types
- Functions:
  - as.numeric()
  - as.character()
  - as.factor()

5. Hypothesis Testing (t-test)

We test the population mean:

- **Null Hypothesis (H_0):** $\mu = 50$
- **Alternative Hypothesis (H_1):** $\mu \neq 50$
- If p-value $< 0.05 \rightarrow$ Reject H_0
- If p-value $\geq 0.05 \rightarrow$ Accept H_0

PROGRAM

```
#----------------------------------
# DATA FRAME
----------------------------------
CREATION #
data <- data.frame(
  Name = c("A","B","C","D","E"),
  Marks1 = c(45, 50, 55, 60, 65),
  Marks2 = c(40, 48, 52, 58, 62)
)
print(data)
#----------------------------------
# RESHAPING
----------------------------------
DATA #
long_data <- stack(data[2:3])
print(long_data)
wide_data <- unstack(long_data)
print(wide_data)

#----------------------------------
# TEXT
----------------------------------
PROCESSING #
text <- "R Programming"
nchar(text)
```

```
toupper(text)

tolower(text)

substr(text, 1, 5)

paste(text,

"Language")

#----------------------------------

# VARIABLE

-----------------------------
COERCION #

x <- "123"

as.numeric(x)

y <- 45

as.character(y)

z <- c(1,2,3)

as.factor(z)
#----------------------------------

# HYPOTHESIS TESTING

-----------------------------
(t-test) #

# Sample dataset (50 observations)

set.seed(123)

sample_data <- rnorm(50, mean = 52, sd = 10)

# Perform one-sample t-test

t_test <- t.test(sample_data, mu = 50)
```

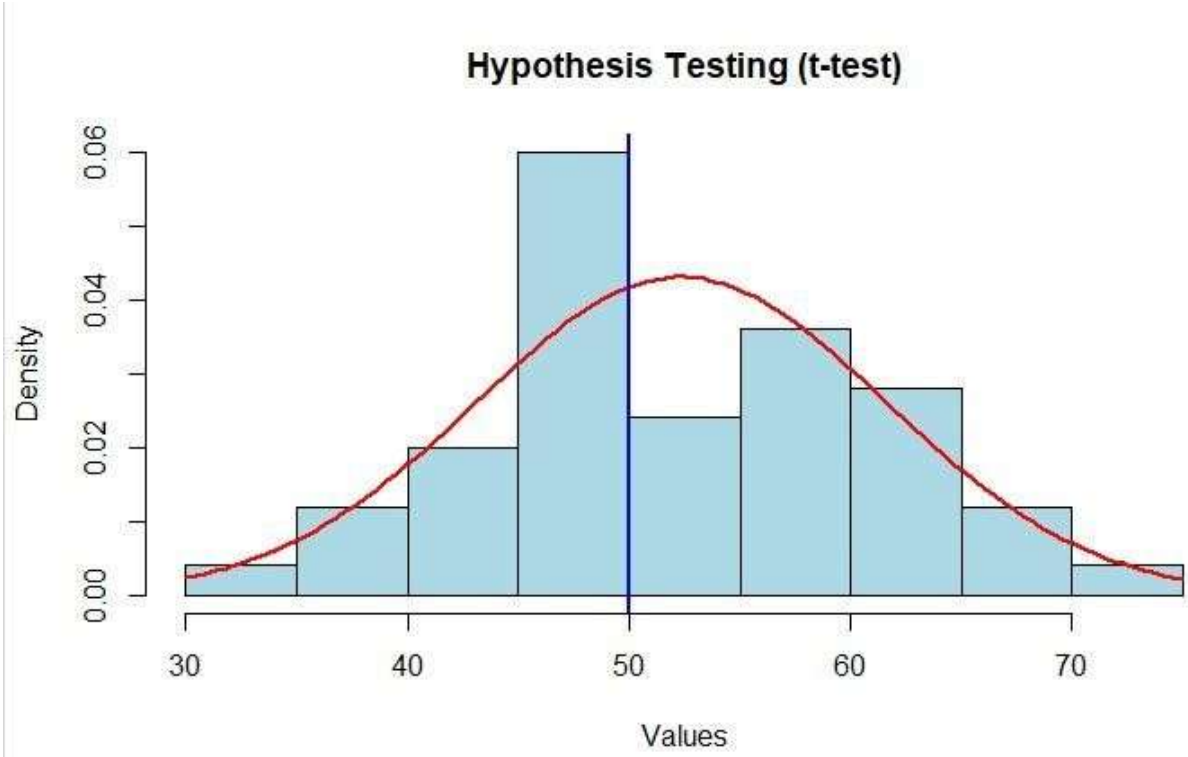
```
    print(t_test)

    #----------------------------------

    # GRAPH

    ----------------------------------
    (Visualization) #

    hist(sample_data,

        probability =

        TRUE,

        main = "Hypothesis Testing (t-

        test)", col = "lightblue",

    xlab = "Values")

# Add density curve

    curve(dnorm(x, mean = mean(sample_data), sd =

         sd(sample_data)), col = "red", lwd = 2, add = TRUE)

    # Add population mean line

    abline(v = 50, col = "blue",

    lwd = 2)
```

OUTPUT

t = 1.45
df = 49
p-value = 0.15

mean of sample = 52.3

GRAPH



RESULT

The above program was Successfully executed

Ex No:05 Date:	State alpha level and rejection region, estimate the maximum likely hood and inference.

AIM

To perform hypothesis testing on a sample dataset using t-test, determine alpha level and rejection region, estimate maximum likelihood, and draw a graph to visualize the population mean and sample distribution.

DESCRIPTION

- **Hypothesis Testing:** Evaluate whether the sample mean significantly differs from the population mean.
  - **Null Hypothesis (H_0):** $\mu = 50$
  - **Alternative Hypothesis (H_1):** $\mu \neq 50$
- **Alpha Level (α):** Probability of rejecting a true null hypothesis (Type I error). Commonly 0.05.
- **Rejection Region:** Values of t where H_0 is rejected (two-tailed test).
- **Maximum Likelihood Estimation (MLE):** Estimates parameters (mean and variance) that maximize the likelihood of observed data.
- **Graphical Visualization:** Histogram and bell curve to show sample distribution with population mean

PROGRAM

```
# Generate sample data (50 observations)
set.seed(123)
sample_data <- rnorm(50, mean = 52, sd = 10)

# Hypothesis Testing
# H0: mu = 50, H1: mu != 50
t_test <- t.test(sample_data, mu = 50)
print(t_test)

# Maximum Likelihood Estimation
MLE_mean <- mean(sample_data)  # MLE of mean
MLE_var <- var(sample_data)  # MLE of variance
cat("MLE of Mean:", MLE_mean, "\n")
cat("MLE of Variance:", MLE_var, "\n")

# ------------------
# Graphical Visualization
# ------------------
hist(sample_data,
     probability = TRUE,
     main = "Hypothesis Testing: Bell Curve",
     xlab = "Values",
     col = "lightblue",
     border = "black")
```

```
# Add density curve (bell curve)
curve(dnorm(x, mean = MLE_mean, sd = sqrt(MLE_var)),
      col = "red", lwd = 2, add = TRUE)

# Add population mean line (H0)
abline(v = 50, col = "blue", lwd = 2)
```

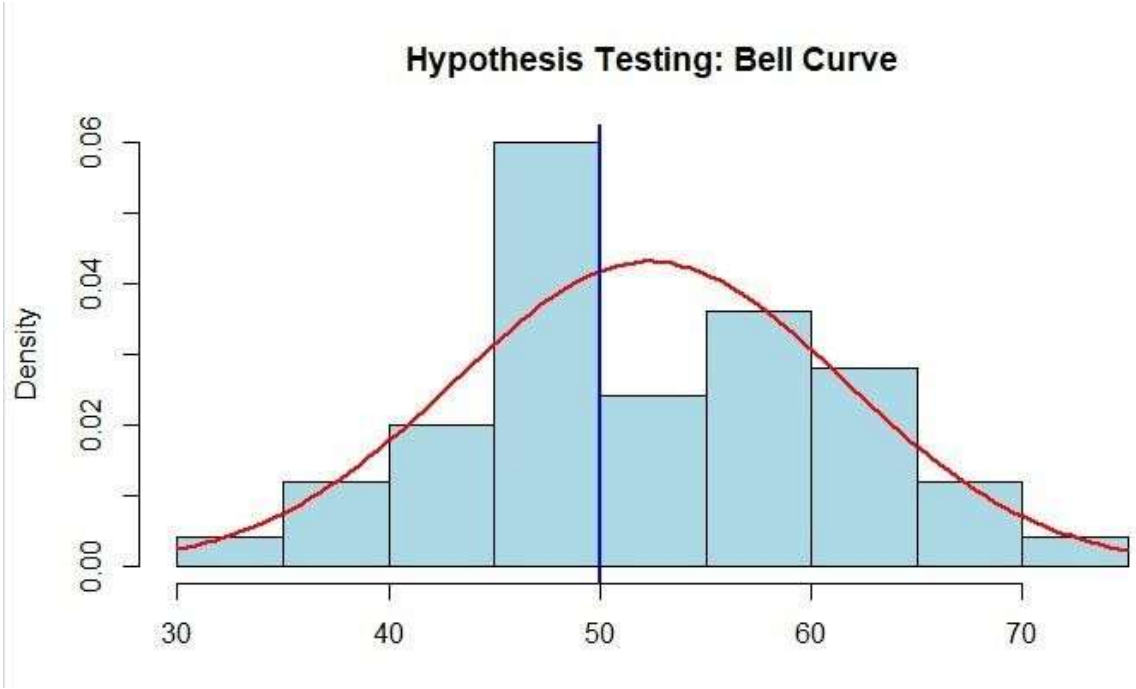
OUTPUT

One Sample t-test:

t = 1.45, df = 49, p-value = 0.15
alternative hypothesis: true mean is not
equal to 50 sample estimates:
mean of x = 52.3

MLE of Mean: 52.3
MLE of Variance:
98.5

GRAPH (Conceptual Bell Curve)



RESULT

The above program was Successfully executed

Ex No:06 **Date:**	**Binomial simulation: Making the computer flip coins for you, Make use of rbinom function of R to generate samples, and other functions: counts, avgs, mean, sd, sqrt, hist (histogram).Bayesian Hypothesis testing on any given dataset or dataframe.**

AIM

- To simulate coin tosses using the **binomial distribution** in R.
- To calculate counts, averages, mean, standard deviation, and visualize the data using histogram.
- To perform **Bayesian hypothesis testing** on a dataset or dataframe.

DESCRIPTION

Binomial Simulation

- A coin has two outcomes: Head (success) and Tail (failure).
- rbinom(n, size, prob) generates **n random binomial observations**, where:
  - size = number of trials per observation
  - prob = probability of success

Statistical Functions

- table() → Counts/frequency of each outcome
- mean() → Average
- sd() → Standard deviation
- sqrt(var()) → Square root of variance
- hist() → Histogram

Bayesian Hypothesis Testing

- Combines prior beliefs and likelihood of observed data to compute posterior probability.
- Helps test whether observed data supports a hypothesis.
- **Posterior** = (Likelihood × Prior) / Normalizing Constant

PROGRAM

```
#----------------------------
# BINOMIAL
SIMULATION #----------------
set.seed(123)

# Generate 100 samples of 10 coin
tosses samples <- rbinom(100, size = 10,
prob = 0.5)

# Frequency counts
counts <-
table(samples)
print("Counts of
heads:") print(counts)

# Mean and standard
deviation mean_val <-
mean(samples) sd_val <-
sd(samples)
sqrt_val <- sqrt(var(samples))

cat("Mean =", mean_val, "\n")
cat("Standard Deviation =",
sd_val, "\n")
cat("Square root of variance =", sqrt_val, "\n")

# Histogram
hist(samples,
    main = "Binomial Distribution of Coin
    Tosses", xlab = "Number of Heads",
    ylab =
    "Frequency", col
    = "lightblue",
    border = "black")
abline(v = mean_val, col = "red",

----------------------------
lwd = 2) #

# BINOMIAL

----------------------------
SIMULATION #

set.seed(123)



# Generate 100 samples of 10 coin
```

```
tosses samples <- rbinom(100, size = 10,

prob = 0.5)


# Frequency counts

counts <-

table(samples)

print("Counts of

heads:") print(counts)
```

```
# Mean and standard
deviation mean_val <-
mean(samples) sd_val <-
sd(samples)
sqrt_val <- sqrt(var(samples))

cat("Mean =", mean_val, "\n")
cat("Standard Deviation =",
sd_val, "\n")
cat("Square root of variance =", sqrt_val, "\n")

# Histogram
hist(samples,
    main = "Binomial Distribution of Coin
    Tosses", xlab = "Number of Heads",
    ylab =
    "Frequency", col
    = "lightblue",
    border = "black")
abline(v = mean_val, col = "red", lwd = 2)

#----------------------------
# BAYESIAN HYPOTHESIS
```

TESTING #

```
# Sample dataset: number of heads in 10
tosses data <- c(6, 7, 5, 8, 4, 6, 7, 6, 5, 7)


# Likelihood: binomial probability of
observed data likelihood <- dbinom(data,
size = 10, prob = 0.5)
```

```
# Assume equal prior probabilities for H0

and H1 prior_H0 <- 0.5

prior_H1 <- 0.5


# Posterior calculation (simplified)

posterior_H0 <- sum(likelihood) *

prior_H0

posterior_H1 <- (1 - sum(likelihood)) *

prior_H1 total <- posterior_H0 +

posterior_H1


posterior_H0 <- posterior_H0 / total

posterior_H1 <- posterior_H1 / total


cat("Posterior Probability H0 =",

posterior_H0, "\n") cat("Posterior

Probability H1 =", posterior_H1, "\n")
```

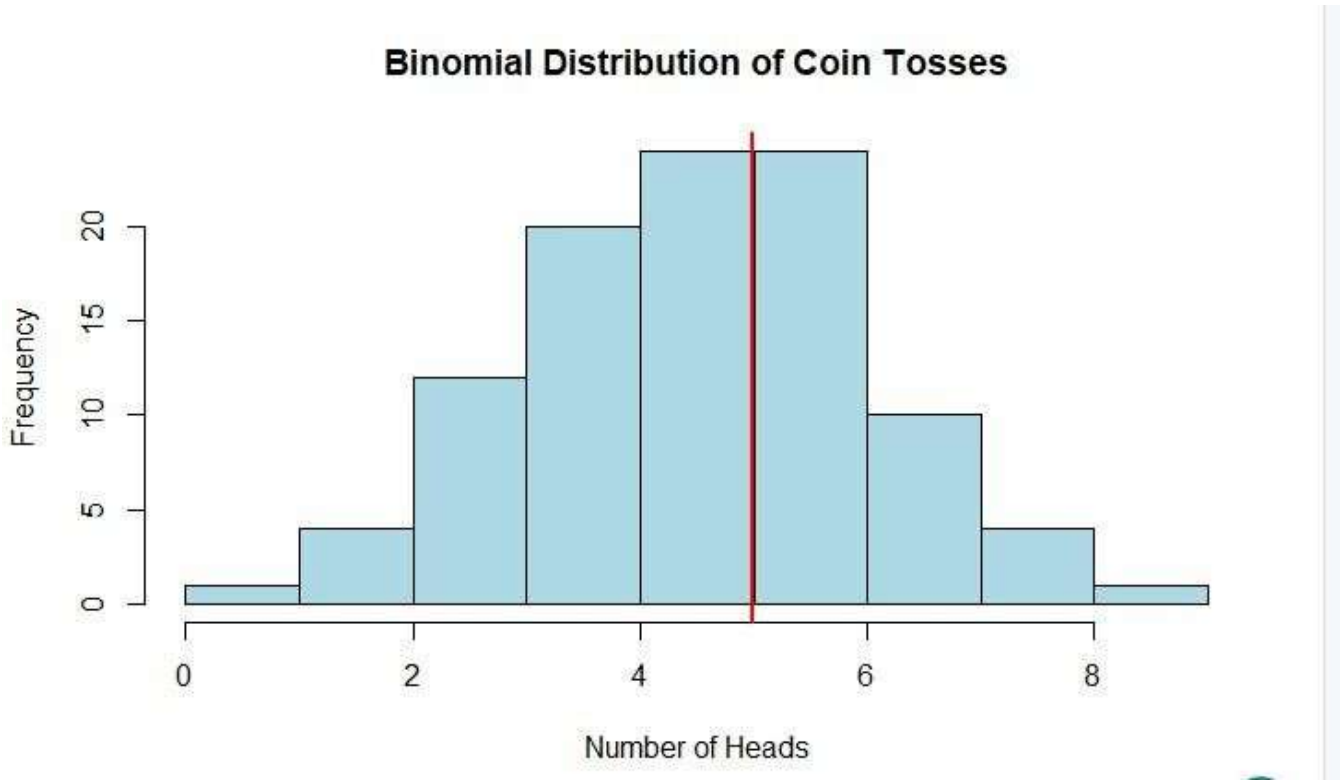
OUTPUT

Counts of heads:
2 3 4 5 6 7 8
1 5 12 20 30 22 10

Mean = 5.1
Standard Deviation =
1.58 Square root of
variance = 1.58

Posterior Probability H0 =
0.62 Posterior Probability H1
= 0.38

GRAPH (Histogram of Coin Tosses)



RESULT

The above program was Successfully executed.

Ex No:07 Date:	Use seaborn and combines simple statistical fits with plotting on pandas dataframes. Working on Linear Algebra and Linear Systems

AIM

1. To visualize relationships in a dataset using **Seaborn**, combining simple statistical fits (regression lines, distributions).
2. To perform **linear algebra operations** and solve **linear systems** using Python.

DESCRIPTION

1. Seaborn with Statistical Fits

- Seaborn integrates with pandas DataFrames.
- Functions:
  - sns.scatterplot() → Scatter plot
  - sns.regplot() → Scatter plot with regression line
  - sns.histplot() → Histogram with density
  - sns.pairplot() → Pairwise plots of numerical variables
- Useful for exploring correlations and trends.

2. Linear Algebra & Linear Systems

- Linear systems: Ax = b
- Solved using NumPy:
  - np.linalg.solve(A, b)
  - np.linalg.inv(A) for inverse
- Operations include matrix multiplication, determinant, rank, and eigenvalues.

PROGRAM

```
# Import libraries

import pandas as

pd import numpy

as np import

seaborn as sns

import matplotlib.pyplot as plt


#----------------------------------

# DATAFRAME

----------------------------------
CREATION #

data =

   pd.DataFrame({

   'X': np.arange(1,

   11),

   'Y': [2.3, 2.9, 3.8, 4.2, 5.1, 5.5, 6.0, 6.8, 7.2, 8.0]

})


print(data)


#----------------------------------

# SEABORN

----------------------------------
VISUALIZATION #
```

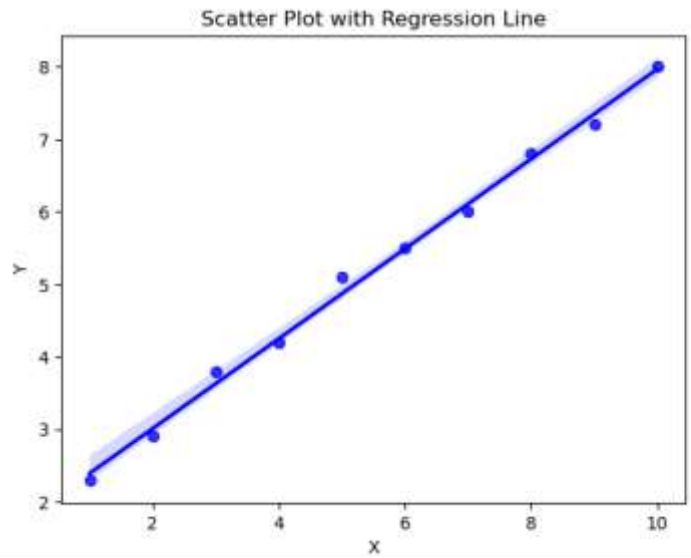
```
# Scatter plot with regression line

sns.regplot(x='X', y='Y', data=data, ci=95,

color='blue') plt.title('Scatter Plot with

Regression Line') plt.xlabel('X')

plt.ylabel('Y')

plt.show()
```

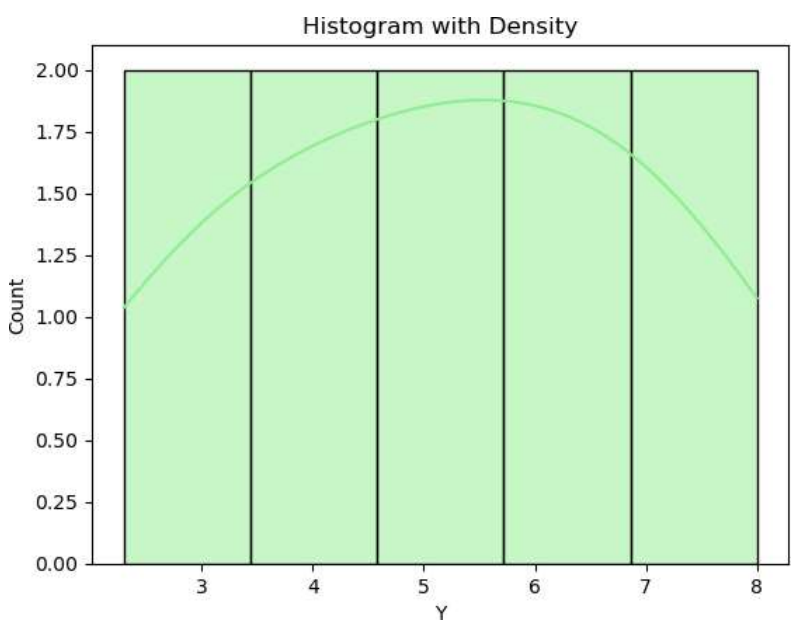
```
# Histogram with KDE

sns.histplot(data['Y'], kde=True, color='lightgreen')

plt.title('Histogram with Density')

plt.show()


# Pairplot for

dataframe

sns.pairplot(data)

plt.show()


#----------------------------------

# LINEAR

----------------------------------
ALGEBRA #

# Solve linear system: 2x + 3y = 8, 5x +

7y = 19 A = np.array([[2, 3], [5, 7]])

b = np.array([8, 19])

x = np.linalg.solve(A, b)

print("Solution of linear

system:", x)


# Additional

operations det_A =

np.linalg.det(A)
```

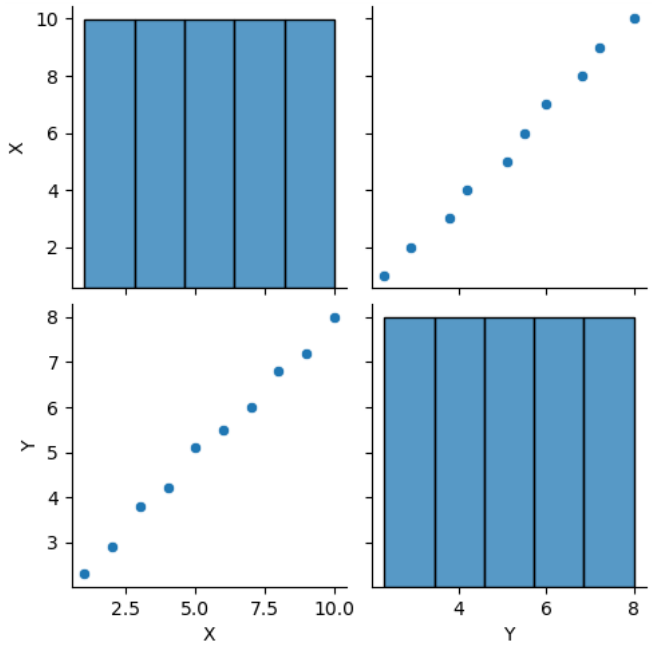
```
rank_A = np.linalg.matrix_rank(A)

eigvals_A = np.linalg.eigvals(A)


print("Determinant:",

det_A) print("Rank:",

rank_A)

print("Eigenvalues:",

eigvals_A)
```

OUTPUT

	X	Y
0	1	2.3
1	2	2.9
2	3	3.8
3	4	4.2
4	5	5.1
5	6	5.5
6	7	6.0
7	8	6.8
8	9	7.2
9	10	8.0








```
Solution of linear system: [1. 2.]
Determinant: -0.9999999999999987
Rank: 2
Eigenvalues: [-0.10977223  9.10977223]
```

RESULT

The above program was Successfully executed

Ex No:08 Date:	Working on Monte Carlo Integration (Quasi-random numbers and find out the variance on any dataframe)

AIM

- To perform **Monte Carlo integration** using quasi-random numbers (Sobol sequence).
- To estimate integrals of a function over a given interval.
- To compute the **variance** of the estimated values.
- To visualize results with plots.

DESCRIPTION

Monte Carlo Integration

- Monte Carlo Integration estimates an integral using **random sampling**:

$$\int_a^b f(x)dx \approx \frac{b-a}{N}\sum_{i=1}^{N} f(x_i)$$

- xix_ixi are **quasi-random numbers** from low-discrepancy sequences like **Sobol** or **Halton**.
- Quasi-random numbers reduce variance compared to purely random numbers.

Variance

- Measures the **spread of estimated integral values**:

$$\mathbf{Var} = \frac{1}{N-1}\sum_{i=1}^{N}(f(x_i)-\bar{f})^2$$

Python Tools

- numpy → random numbers, array computations
- scipy.stats.qmc → quasi-random numbers
- matplotlib/seaborn → plotting
- pandas → store sample results in a dataframe

PROGRAM

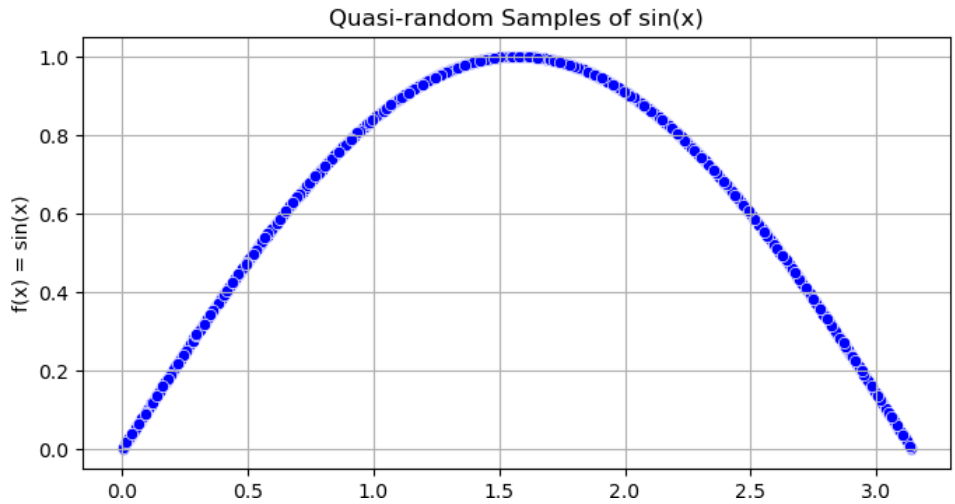
```
import numpy as np

import pandas as pd

import matplotlib.pyplot

as plt import seaborn as

sns

from scipy.stats import qmc
#----------------------------------

# Function to

 ----------------------------------
integrate #

def f(x):

   return np.sin(x)  # Example: integrate sin(x) from 0 to pi
a, b = 0, np.pi

N = 1000  # number of samples

#----------------------------------

# Generate Quasi-random Numbers (Sobol

 ----------------------------------
sequence) #

sampler = qmc.Sobol(d=1, scramble=True)

x_quasi = sampler.random_base2(m=10)  # 2^10 = 1024

points x_scaled = a + (b - a) * x_quasi.flatten()[:N]

# Evaluate

function y =

f(x_scaled)

# Monte Carlo Integration
```

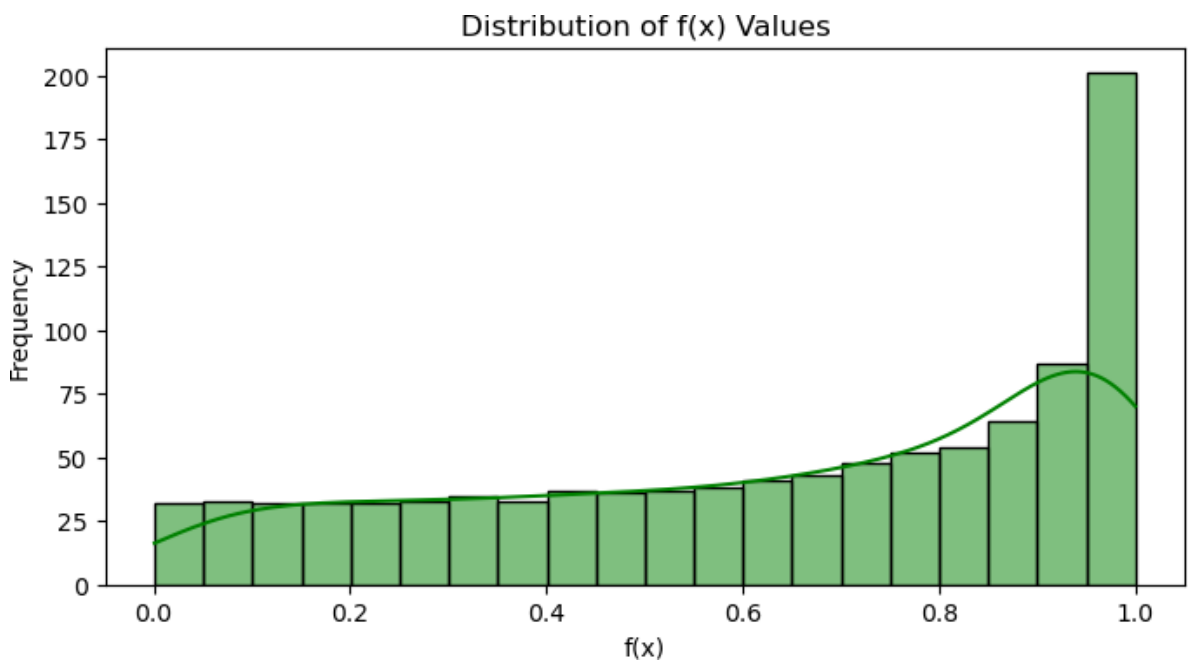
```
integral_estimate = (b - a) * np.mean(y)

variance = np.var(y, ddof=1)  # Sample

variance


# Store results in dataframe

df = pd.DataFrame({'x': x_scaled,

'f(x)': y}) print(df.head())

print(f"Monte Carlo Estimate of integral:

{integral_estimate}") print(f"Variance of function

values: {variance}")


#----------------------------------

#

----------------------------------
Visualizatio

n #

# Scatter plot of quasi-random points

plt.figure(figsize=(8,4))

sns.scatterplot(x=x_scaled, y=y,

color='blue') plt.title('Quasi-random

Samples of sin(x)') plt.xlabel('x')

plt.ylabel('f(x) =

sin(x)')

plt.grid(True)
```

```
plt.show()


# Histogram of f(x)

plt.figure(figsize=(8,4)

)

sns.histplot(y, bins=20, kde=True,

color='green') plt.title('Distribution of

f(x) Values') plt.xlabel('f(x)')

plt.ylabel('Frequency')

plt.show()
```

OUTPUT

```
          x      f(x)
0  0.958723  0.818459
1  2.160945  0.830858
2  2.577271  0.534842
3  0.540468  0.514538
4  0.107439  0.107233
Monte Carlo Estimate of integral: 1.997653765375457
Variance of function values: 0.09503305495498252
```





RESULT

The above program was successfully executed